

Automotive Security

Protecting a 2-tonne computer that drives at 120 km/h

"In a phone, a bug leaks your photos. In a car, a bug can steer."

- attacks that really happened
- standards you must name in interviews
- secure coding checklist for AOS apps

A Head First-style learning guide that goes with the **Automotive Academy** sample app (Android Automotive OS, Kotlin, Jetpack Compose). Generated September 2026.

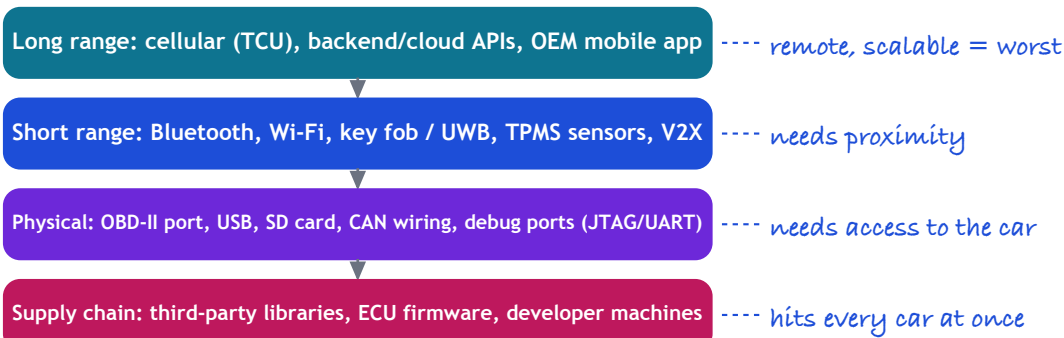
What's in this book

1 Why car security is different	3
Attack surfaces	3
2 Hacks that changed the industry	4
3 Standards and regulations	5
TARA: Threat Analysis and Risk Assessment (ISO/SAE 21434)	5
4 Android platform security on AAOS	6
Verified boot: the chain of trust	6
Permissions: four protection levels	6
5 In-vehicle network security	7
SecOC: Secure Onboard Communication (AUTOSAR)	7
6 Keys, HSMs and the digital key	8
7 OTA security	8
8 Secure coding checklist for AAOS app developers	9
9 Privacy	9
10 Security questions interviewers ask	10
Sharpen your pencil: answers	11

1 Why car security is different

Phone	Car
Replaced every 2-4 years	On the road 15-20 years; crypto must age well
Worst case: data theft	Worst case: physical harm (safety + security are linked)
One vendor controls most of the stack	Dozens of suppliers, ECUs and firmware images
Rarely physically attacked	Attacker can touch the OBD port, bus wiring, key fob radio
Updates are easy	Updates must never brick; some ECUs cannot be updated OTA at all

Attack surfaces



BRAIN POWER

Which attack surface scares an OEM most, and why? Think about *scale*: one exploit vs one car.

2 Hacks that changed the industry

learn from other people's recalls

Year	What happened	Lesson
2015	Charlie Miller & Chris Valasek remotely controlled a Jeep Cherokee (steering, brakes at low speed) via the Uconnect head unit's cellular connection. FCA recalled ~1.4 million vehicles.	Head unit must not be able to reach safety buses; add a secure gateway; firewall the modem.
2015	ADAC found BMW ConnectedDrive used unencrypted HTTP; doors could be unlocked by a fake base station. ~2.2 million cars patched over the air.	Use TLS with certificate validation everywhere.
2016	Keen Security Lab remotely attacked a Tesla Model S through the browser/Wi-Fi and reached CAN.	Browser/rendering engines are attack surface; code signing for firmware; fast OTA response.
2016	NissanConnect EV API allowed controlling climate of other Leafs knowing only the VIN.	Backend APIs need real authentication; VINs are not secrets.
2022-2024	Researchers (e.g. Sam Curry et al.) found OEM web portals/APIs that allowed locating, unlocking or starting cars of many brands; a 2024 Kia issue needed only the licence plate.	The cloud is part of the vehicle attack surface.
Ongoing	Relay attacks on keyless entry: two radios extend the key fob range, the car opens.	Distance bounding with UWB (CCC Digital Key 3.0); motion-sensing fobs.

INTERVIEW TIP

Tell the Jeep story in 30 seconds and end with the architecture lesson: 'That's why the IVI is isolated behind a secure gateway and why CAN messages to safety ECUs need SecOC.'

3 Standards and regulations

names you must drop in the interview

Standard	What it requires	Since
UNECE R155	Cybersecurity Management System (CSMS) for the OEM organisation + vehicle type approval	EU: new types July 2022, all new vehicles July 2024
UNECE R156	Software Update Management System (SUMS), safe & secure OTA	Same timeline as R155
ISO/SAE 21434	Cybersecurity engineering across the lifecycle: TARA, requirements, verification, incident response	Published 2021
ISO 26262	Functional safety (not security, but they must be coordinated)	2011, 2nd ed. 2018
ISO 24089	Software update engineering	2023
China GB 44495-2024	Mandatory vehicle cybersecurity requirements for China	Applies from 2026

FIRESIDE CHAT

Tonight's talk: Functional Safety vs Cybersecurity

Safety (ISO 26262): I worry about things going wrong by accident: a sensor fails, a bit flips, a bug in the braking code. I ask: 'what if this fails?'

Security (ISO/SAE 21434): I worry about someone making things go wrong on purpose. I ask: 'what if someone attacks this?'

Safety: My failures follow statistics. Yours don't: attackers pick the worst moment.

Security: And if I lose, you lose too. A spoofed brake message is my problem and your hazard.

Both: That's why OEMs run HARA and TARA side by side and share findings.

TARA: Threat Analysis and Risk Assessment (ISO/SAE 21434)



SHARPEN YOUR PENCIL

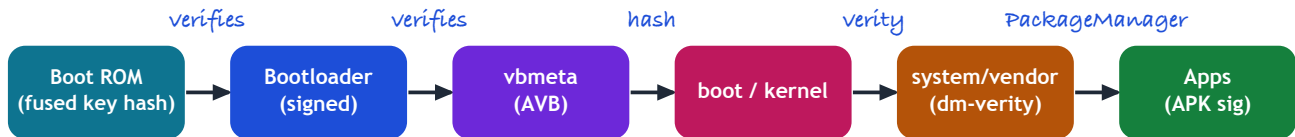
Do a mini-TARA for the asset 'unlock doors' CAN message:

1. Damage scenario? 2. STRIDE category? 3. One attack path? 4. A treatment?

(Answers at the end of the book.)

4 Android platform security on AAOS

Verified boot: the chain of trust



Each stage verifies the next *before* running it. The root of trust is in hardware (ROM + fuses). AVB also stores a **rollback index** so an attacker cannot install an older, vulnerable, but properly signed image.

SEE IT MOVE: IN THE APP

Lesson **13 Security** → 'Tamper with kernel & boot': the chain ticks green until the kernel, turns red and halts. Tampered code never runs.

Permissions: four protection levels

Level	Granted how	Car examples
normal	At install, automatically	CAR_INFO, CAR_POWERTRAIN, CAR_EXTERIOR_ENVIRONMENT
dangerous	User dialog at runtime	CAR_SPEED, CAR_ENERGY (Android 14+), location, microphone
signature	Only apps signed with the same key as the definer (platform/OEM)	Many CarService internals
privileged (signature privileged)	Signature OR pre-installed in /system/priv-app + allowlisted in privapp-permissions XML	CONTROL_CAR_CLIMATE, CONTROL_CAR_DOORS, CAR_VENDOR_EXTENSION

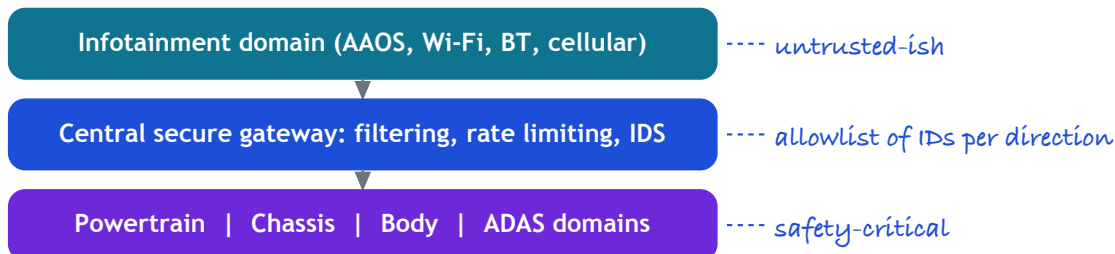
- **SELinux** (enforcing) confines every process, including the VHAL and CarService, to a policy domain.
- **App sandbox**: each app has its own Linux UID; each Android user has separate storage.
- **Keystore / TEE / StrongBox**: keys generated and used inside secure hardware; the app only gets handles.
- **Seccomp, ASLR, CFI, hardened malloc**: exploit mitigations inherited from Android.

```
// core/security/SecureVault.kt - AES-256-GCM key that never leaves the Keystore
KeyGenParameterSpec.Builder(alias, PURPOSE_ENCRYPT or PURPOSE_DECRYPT)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
    .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
    .setKeySize(256)
    .build()
```

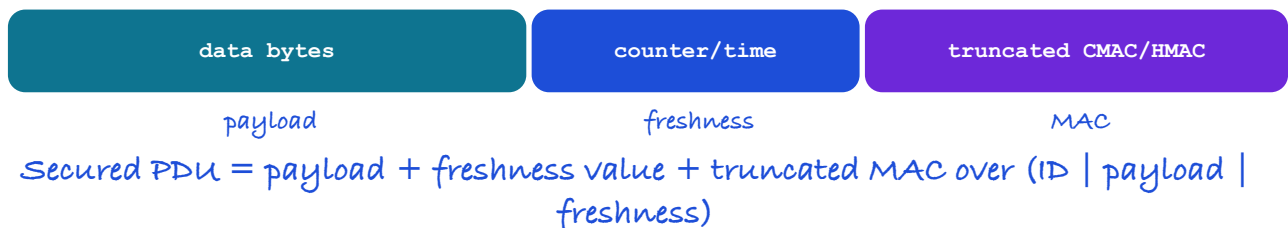
SEE IT MOVE: IN THE APP

Security lesson → 'Encrypt key token': a digital key token is encrypted with AES-GCM, stored as Base64 and decrypted. The app reports whether the key is inside secure hardware.

5 In-vehicle network security



SecOC: Secure Onboard Communication (AUTOSAR)



Attack	What SecOC does
Spoofing (send fake 'unlock')	Receiver recomputes MAC with the shared key; mismatch → drop
Tampering (change payload)	MAC no longer matches → drop
Replay (resend yesterday's valid frame)	Freshness value must be newer than the last accepted one → drop

SEE IT MOVE: IN THE APP

Security lesson → SecOC demo: 'Legit unlock' is ACCEPTED, 'Attacker spoof' is REJECTED_BAD_MAC, 'Replay old frame' is REJECTED_REPLAY. The logic is in SecOc.kt and verified by SecOcTest.

WATCH IT!

Truncating the MAC (e.g. to 24-64 bits) is normal on CAN because space is tiny; security then also depends on the receiver limiting the attempt rate.

Key management is the hard part: keys are provisioned in the factory into each ECU's HSM.

- **IDS**: detect unusual frame rates, unknown IDs, impossible signal values; report to a Vehicle SOC in the cloud.
- **Ethernet**: VLAN segmentation, MACsec (802.1AE), TLS/IPsec for SOME/IP and diagnostics.

- **Secure diagnostics:** UDS 0x27 SecurityAccess (seed/key, weak if the algorithm leaks) is being replaced by 0x29 Authentication (certificate-based, PKI).

6 Keys, HSMs and the digital key

Component	Role
HSM (in ECU microcontrollers, e.g. EVITA levels)	Stores keys, does AES-CMAC for SecOC, secure boot of the MCU
TEE / StrongBox (in the Android SoC)	Android Keystore keys, DRM, biometric templates
Backend PKI	Issues certificates for vehicles, ECUs, V2X, diagnostic testers
CCC Digital Key 3.0	Phone as key over NFC + BLE + UWB; UWB time-of-flight defeats relay attacks

BRAIN POWER

A relay attacker forwards radio signals between your key (in your house) and your car. Why can BLE signal strength be fooled, but UWB time-of-flight cannot be made *shorter*?

7 OTA security

- Packages are **signed** by the OEM; the device verifies before installing (and AVB verifies again at boot).
- **Anti-rollback:** rollback index prevents installing older vulnerable images.
- **Uptane:** separates a Director repository (what this car should install) from an Image repository (what exists) with separate keys, so a single compromised server cannot push malware to the fleet.
- Transport over TLS with pinned certificates; resume + integrity checks for partial downloads.
- Install only when safe (parked), with user consent, logged for R156 audits.

SEE IT MOVE: IN THE APP

Lesson **12 OTA** shows verify → install inactive slot → reboot → rollback if the new slot fails to boot.

8 Secure coding checklist for AAOS app developers

[print this page](#)

Area	Do	Don't
Exported components	Set android:exported explicitly; protect services with permissions	Leave services exported without checks
MediaBrowserService	Validate caller in onGetRoot (package + signature); return a limited root for unknown callers	Expose the full library to any app
Car App Library	HostValidator allowlist in release	Ship ALLOW_ALL_HOSTS_VALIDATOR
Permissions	Request only what you need; handle SecurityException	Assume car permissions are granted
Secrets	Android Keystore (AES-GCM), no keys in APK	Hard-code API keys or tokens
Network	TLS + network security config; certificate pinning for sensitive APIs	Cleartext HTTP
Multi-user	Store data per Android user; wipe guest data	Assume user 0 or share data across drivers
Logging	Strip PII/location/VIN from logs in release	Log tokens or precise location
Intents	Use explicit intents; validate extras	Trust data in incoming intents
Dependencies	SBOM, vulnerability scanning, pin versions	Pull random SDKs into a car app

9 Privacy

Cars know where you live, work and sleep, how you drive, who you call, and (with driver-monitoring cameras) where you look. GDPR (EU) and DPDP Act 2023 (India) apply. Principles: data minimisation, purpose limitation, consent, retention limits, and per-user separation (a guest must not see the owner's contacts or history).

10 Security questions interviewers ask

THERE ARE NO DUMB QUESTIONS

Q: What is the difference between safety and security?

A: Safety (ISO 26262) protects against *random failures and systematic errors*; security (ISO/SAE 21434) protects against *intentional attackers*. A security breach can cause a safety hazard, so the two analyses are linked.

Q: Why can't we just encrypt CAN?

A: Classic CAN has 8 bytes and hard real-time deadlines; encryption adds latency and doesn't solve authentication alone. SecOC adds authenticity + freshness; CAN FD gives room for MACs. Confidentiality is usually not the goal for control messages.

Q: How does AAOS stop a random app from unlocking the doors?

A: CONTROL_CAR_DOORS is signature|privileged; CarService checks it before calling VHAL; SELinux prevents the app from talking to VHAL directly; the gateway and SecOC protect the bus side.

Q: What is a TARA?

A: Threat Analysis and Risk Assessment: identify assets, damage scenarios, threats, attack paths, rate impact and feasibility, compute risk, choose treatment. Required by ISO/SAE 21434 and audited under R155.

Q: What is a root of trust?

A: An immutable starting point (Boot ROM + fused public key hash, or an HSM) that is trusted by definition and verifies everything after it.

Q: How do you store a refresh token in a car app?

A: Encrypt it with an Android Keystore AES-GCM key (hardware-backed when available), per Android user; never in plain SharedPreferences or in the APK.

Sharpen your pencil: answers

- Damage scenario: an attacker unlocks the car → theft or harm to occupants.
- STRIDE: **S**poofing (fake sender) and **T**ampering; replay is a spoofing variant.
- Attack path: compromise IVI via Bluetooth bug → send CAN frame through a weak gateway rule; or plug into the bus behind the headlight.
- Treatment: SecOC with freshness on the door message, gateway allowlist (IVI may not send `0x2Ao`), IDS alerts.

BULLET POINTS

- Cars: long life, physical access, safety impact, many suppliers.
- Remember Jeep 2015 → isolate IVI with a gateway.
- R155 (CSMS), R156 (SUMS), ISO/SAE 21434 (TARA).
- Verified boot + SELinux + permissions + Keystore on AAOS.
- SecOC = MAC + freshness against spoofing and replay.
- UWB distance bounding against relay attacks.